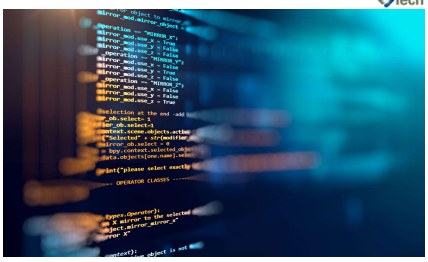
Python Programming Foundations

Week 4 – Debugging, Testing, & Reading Structured Code

Day 1: "Debugging as Evidence Gathering"

1



Review Coding Lab

2

A Bug Is Information

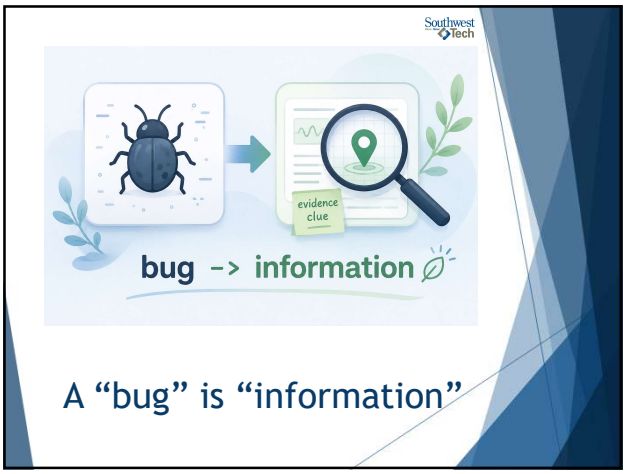
A bug is information, not a verdict.

When code breaks or gives the wrong answer, the program is giving you clues.

Today, practice debugging by asking:

- What did I expect?
- What actually happened?
- Where is the first useful clue?
- What fix does the evidence justify?

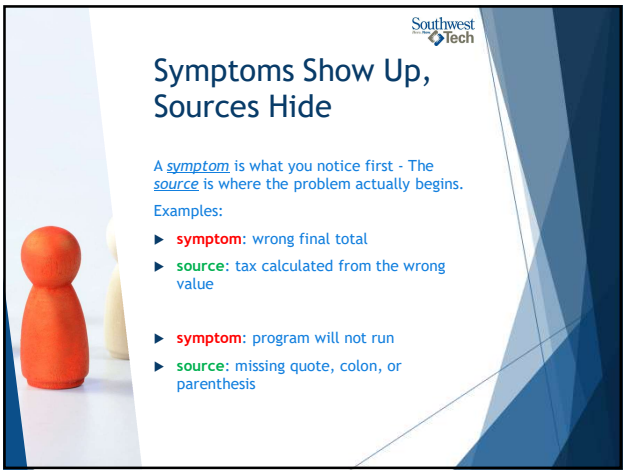
3


Southwest Tech
evidence clue
bug -> information
A “bug” is “information”

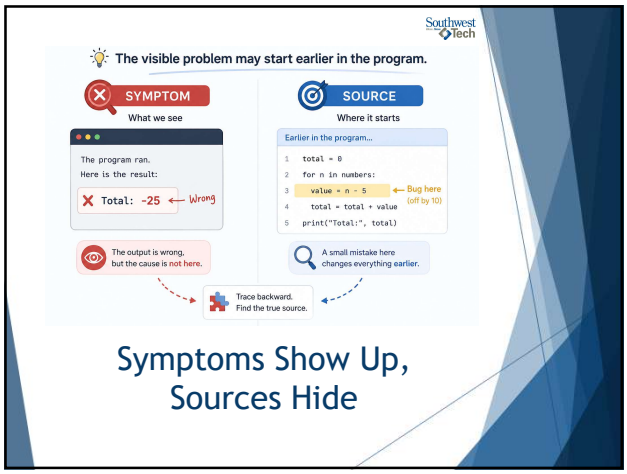
4


Southwest Tech
Build Becomes Inspect
In earlier weeks, you built small programs.
Now you inspect behavior when a program does not do what it should.
Debugging uses familiar skills:
- reading code
- running code
- tracing values
- explaining output

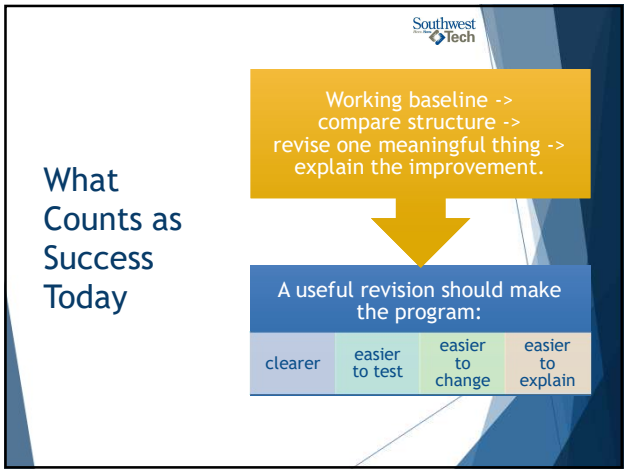
5


Southwest Tech
Symptoms Show Up, Sources Hide
A symptom is what you notice first - The source is where the problem actually begins.
Examples:
symptom: wrong final total
source: tax calculated from the wrong value
symptom: program will not run
source: missing quote, colon, or parenthesis

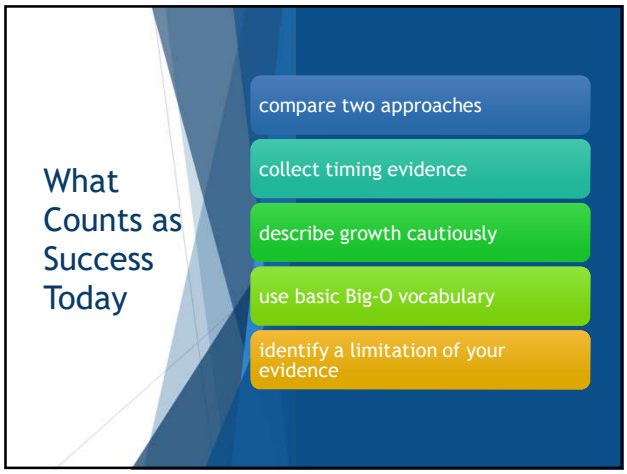
6


Southwest Tech
The visible problem may start earlier in the program.
SYMPTOM
What we see
SOURCE
Where it starts
The program ran.
Here is the result:
Total: -25
Wrong
Earlier in the program...
total = 0
for n in numbers:
value = n - 5
Bug here (off by 10)
total = total + value
print("Total:", total)
The output is wrong, but the cause is not here.
A small mistake here changes everything earlier.
Trace backward. Find the true source.
Symptoms Show Up, Sources Hide

7


Southwest Tech
What Counts as Success Today
Working baseline -> compare structure -> revise one meaningful thing -> explain the improvement.
A useful revision should make the program:
clearer
easier to test
easier to change
easier to explain

8


What Counts as Success Today
compare two approaches
collect timing evidence
describe growth cautiously
use basic Big-O vocabulary
identify a limitation of your evidence

9


Southwest Tech
What We Will Use Today
expected versus actual
tracebacks
labeled print-debugging
one-change-at-a-time repair
simple evidence notes
These tools help you investigate before changing code.

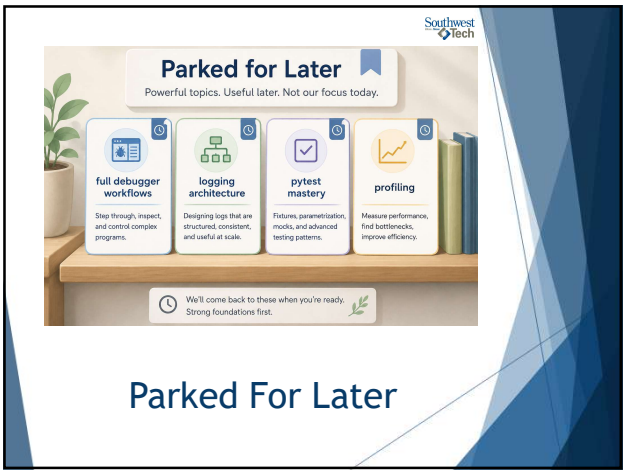
10


Southwest Tech
Debugging Toolbox
Small tools. Clear thinking. Better programs.
expected vs actual
traceback clue
labeled print
one-change repair
evidence notes
Debugging is not guessing. It's collecting clues, making small changes, and learning.
Toolbox

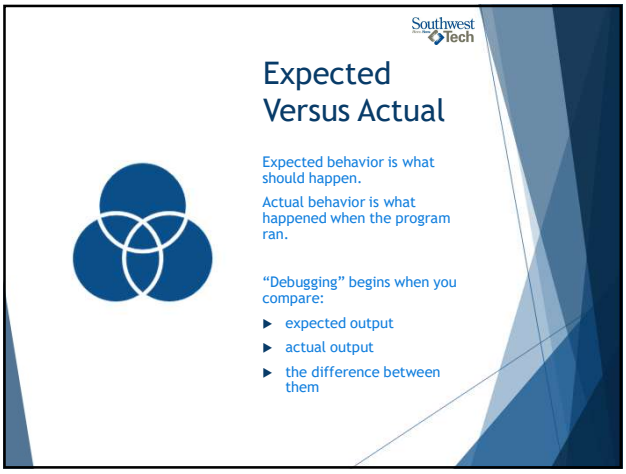
11


Southwest Tech
What We Will "Skip" For Now
full debugger workflows
logging architecture
pytest mastery
profiling for now.
Today's required target is smaller:
find useful evidence
make a justified fix
explain why the fix works

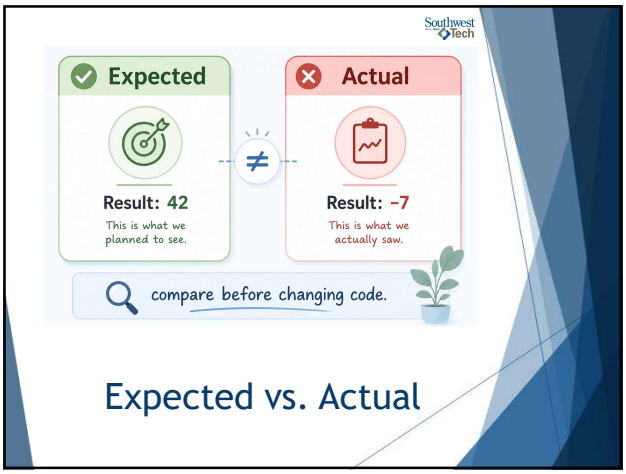
12


Southwest Tech
Parked for Later
Powerful topics. Useful later. Not our focus today.
full debugger workflows
Step through, inspect, and control complex programs.
logging architecture
Designing logs that are structured, consistent, and useful at scale.
pytest mastery
Fixtures, parametrization, mocks, and advanced testing patterns.
profiling
Measure performance, find bottlenecks, improve efficiency.
We'll come back to these when you're ready. Strong foundations first.
Parked For Later

13


Southwest Tech
Expected Versus Actual
Expected behavior is what should happen.
Actual behavior is what happened when the program ran.
"Debugging" begins when you compare:
expected output
actual output
the difference between them

14


Southwest Tech
Expected
Result: 42
This is what we planned to see.
≠
Actual
Result: -7
This is what we actually saw.
compare before changing code.
Expected vs. Actual

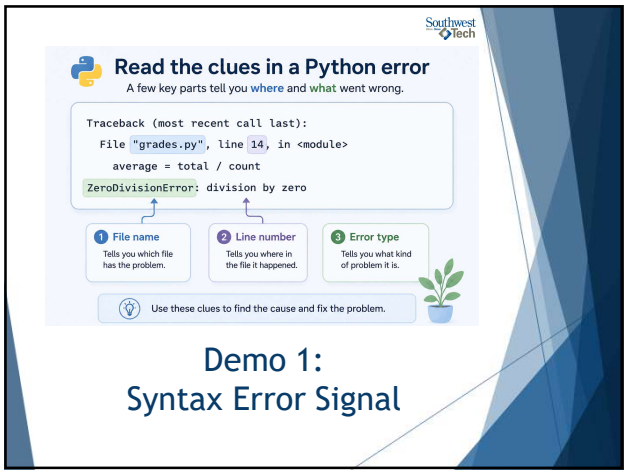
15


Demo #1
"Broken Syntax Example"

16


Southwest Tech
What to Watch For
A syntax error prevents Python from running the program.
Read the message for:
file name
line number
error type
the nearby code Python could not understand
Q2
Q3

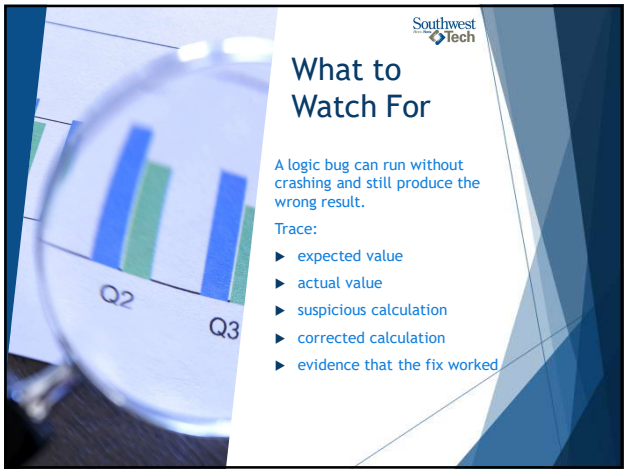
17


Southwest Tech
Read the clues in a Python error
A few key parts tell you where and what went wrong.
Traceback (most recent call last):
File "grades.py", line 14, in <module>
average = total / count
ZeroDivisionError: division by zero
1 File name
Tells you which file has the problem.
2 Line number
Tells you where in the file it happened.
3 Error type
Tells you what kind of problem it is.
Use these clues to find the cause and fix the problem.
Demo 1:
Syntax Error Signal

18


Demo #2
"Basic Type Awareness"

19


Southwest Tech
What to Watch For
A logic bug can run without crashing and still produce the wrong result.
Trace:
expected value
actual value
suspicious calculation
corrected calculation
evidence that the fix worked
Q2
Q3

20


Southwest Tech
Discount Calculation
Item price: $100
Discount: 20%
Expected
$80.00
100 − 20% of 100 = 80
Actual
$120.00
This doesn't look right.
Let's find where the calculation went wrong.
Calculation Steps
1 Start with item price 100.00
2 Calculate discount (20% of 100) 20.00
3 Apply discount to price 120.00
price = price + discount
4 Tax (0%) 0.00
Investigate here
5 Final price 120.00
The discount should be subtracted, not added.
price = price − discount
Demo 2: Logic Bug
Expected vs Actual

21

Ask One Question, Print One Clue

Print-debugging should answer a specific question.

Weak

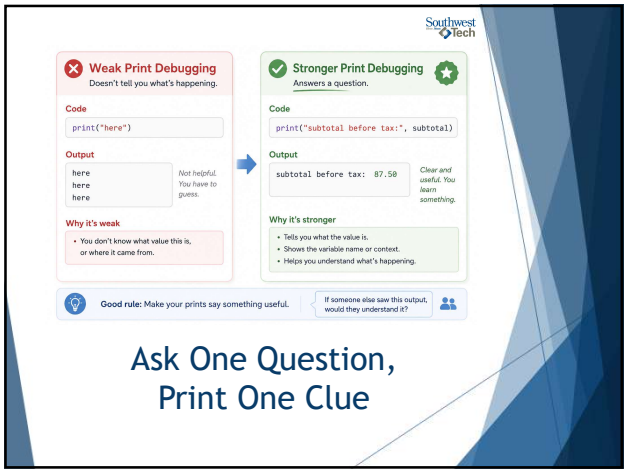
```
Code_Sample.py
1
2  print("here")
3
```

Better

```
2
3  print("subtotal before tax:", subtotal)
4
```

Every debug print should reveal a value or path you need to inspect.

22

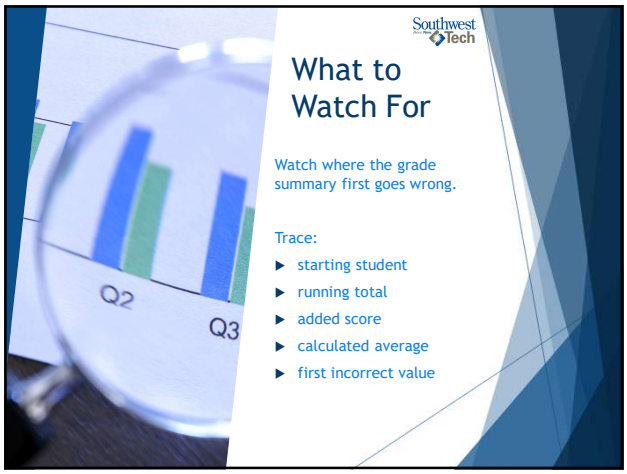


23

Demo #3

"Grade Summary Debugging"

24



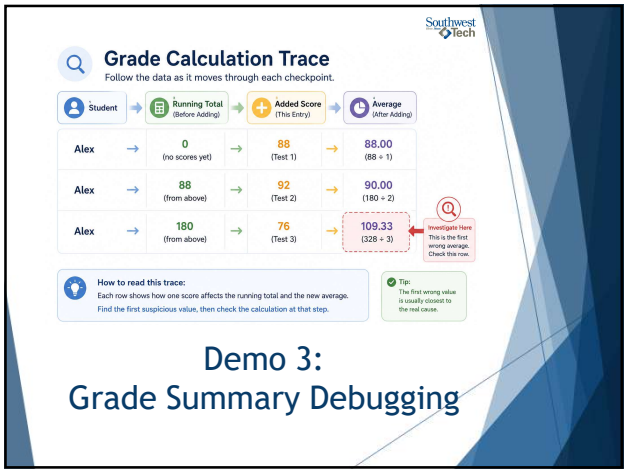
What to Watch For

Watch where the grade summary first goes wrong.

Trace:

- starting student
- running total
- added score
- calculated average
- first incorrect value

25



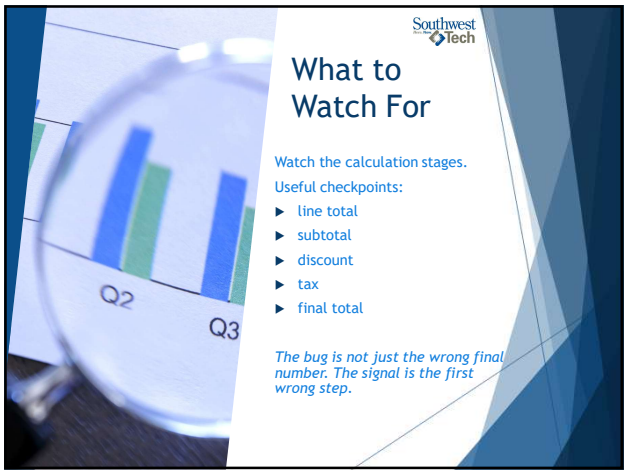
Demo 3: Grade Summary Debugging

26

Demo #4

"Order Total Debugging"

27



What to Watch For

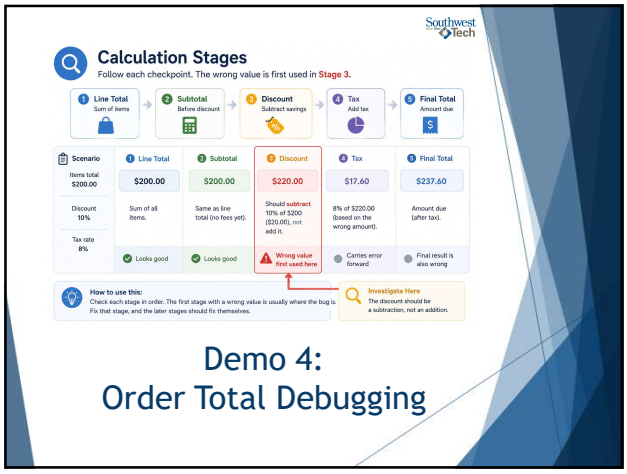
Watch the calculation stages.

Useful checkpoints:

- line total
- subtotal
- discount
- tax
- final total

The bug is not just the wrong final number. The signal is the first wrong step.

28



Demo 4: Order Total Debugging

29

Evidence Before Repair

Good debugging evidence narrows the problem.

Evidence may be:

- a traceback line
- expected versus actual output
- a labeled debug print
- the first wrong value
- a simple check after the fix

*Do **not** change several things* before you know *what* the evidence says.

30

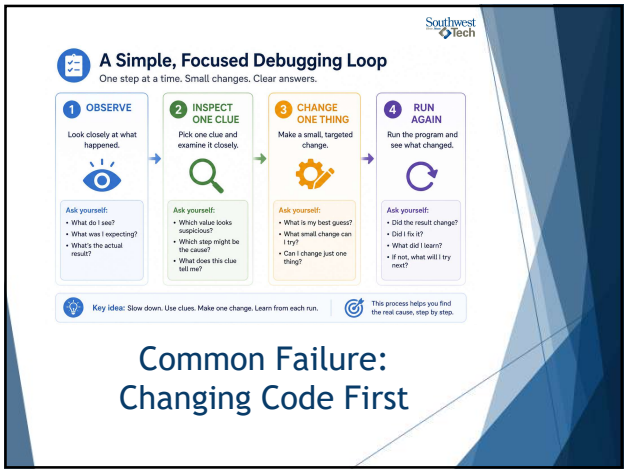
Common Failure: Changing Code First

Changing code before observing can destroy the trail.

Instead:

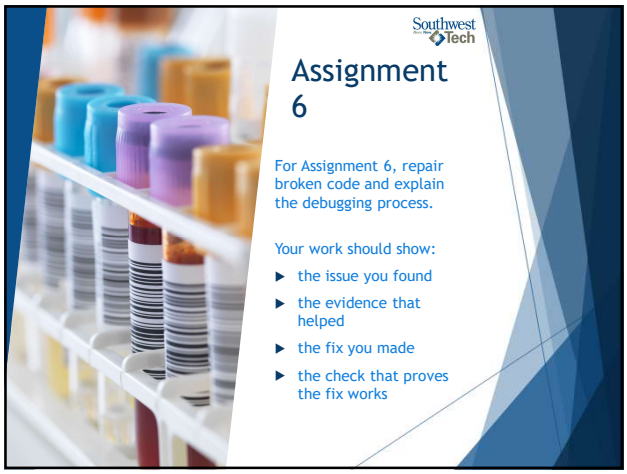
- reproduce the problem
- record what happened
- inspect one clue
- change one thing
- run again

31



Common Failure: Changing Code First

32



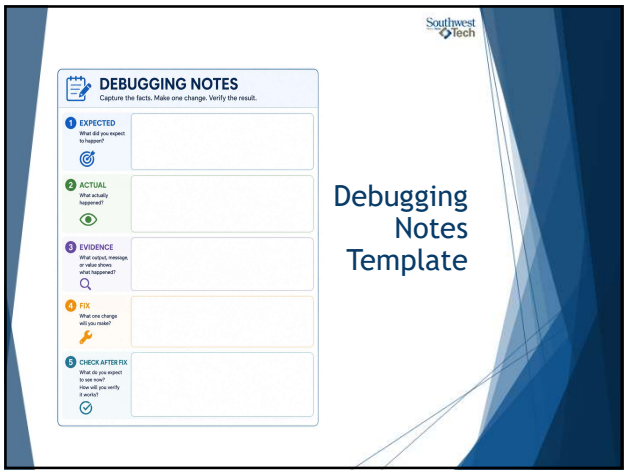
Assignment 6

For Assignment 6, repair broken code and explain the debugging process.

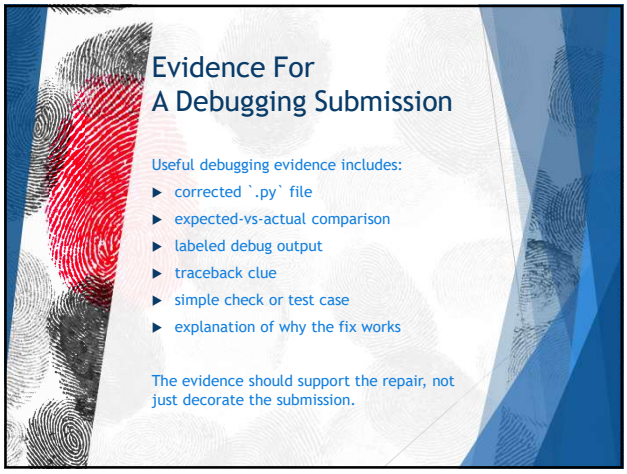
Your work should show:

- the issue you found
- the evidence that helped
- the fix you made
- the check that proves the fix works

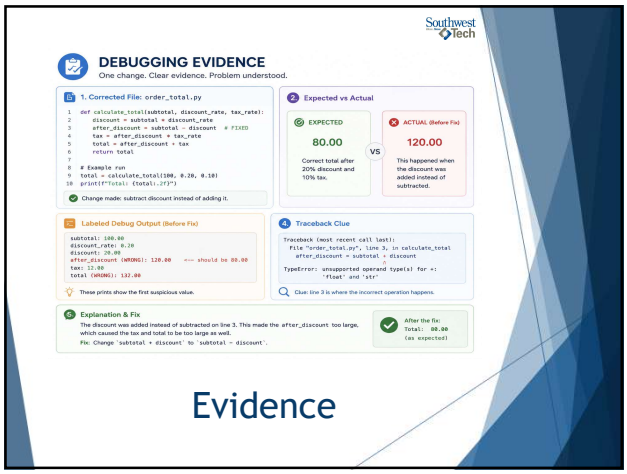
33


Southwest Tech
DEBUGGING NOTES
Capture the facts. Make one change. Verify the result.
1 EXPECTED
What did you expect to happen?
2 ACTUAL
What actually happened?
3 EVIDENCE
What output, message, or value shows what happened?
4 FIX
What one change will you make?
5 CHECK AFTER FIX
What do you expect to see now?
How will you verify it works?
Debugging Notes Template

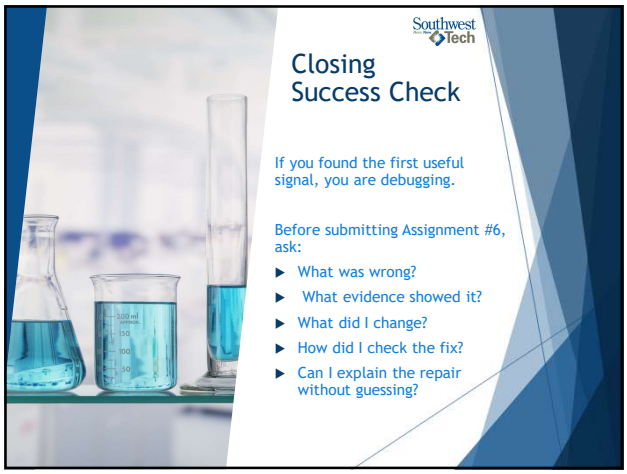
34


Evidence For A Debugging Submission
Useful debugging evidence includes:
corrected `.py` file
expected-vs-actual comparison
labeled debug output
traceback clue
simple check or test case
explanation of why the fix works
The evidence should support the repair, not just decorate the submission.

35


Southwest Tech
DEBUGGING EVIDENCE
One change. Clear evidence. Problem understood.
1. Corrected File: order_total.py
2 Expected vs Actual
EXPECTED
80.00
ACTUAL (Before Fix)
120.00
Labeled Debug Output (Before Fix)
4 Traceback Clue
5 Explanation & Fix
Evidence

36


Southwest Tech
Closing
Success Check
If you found the first useful signal, you are debugging.
Before submitting Assignment #6, ask:
What was wrong?
What evidence showed it?
What did I change?
How did I check the fix?
Can I explain the repair without guessing?

37


Southwest Tech
Thank you!
Have Fun!

38